

MagicCode Super-Resolution Software FAQ and Troubleshooting

Document Version: v2.1.0. Release Date: September 4, 2026.

1. General

What does MagicCode Super-Resolution do?

It upscales images or GPU frames. v2.1.0 public modes are SPATIAL_SPEED_MODE, SPATIAL_BALANCED_MODE, TEMPORAL_SPEED_MODE, and TEMPORAL_BALANCED_MODE.

What functions are exported in v2.1.0?

Exactly 3 functions: MC_Enable, MC_Disable, and MC_GetVersion.

Does the software upload my images or videos?

No. Technical reporting does not upload image or video content. See the Privacy Policy and Data Reporting Notice (unchanged by this technical release).

What scale factors are supported?

Spatial: [1.0, 8.0]. Temporal: (1.0, 8.0]. Actual availability depends on backend and model. API support is not a claim that every scale is device-run.

2. Temporal FAQ

How is temporal called in v2.1.0?

Through MC_Enable with alg_mode set to TEMPORAL_SPEED_MODE or TEMPORAL_BALANCED_MODE, and magic_frame_t.frame pointing to a populated temporal_frame_t.

If I omit masks, do I have a semantic transparency mask?

No. The library derives an approximate mask when inputs are missing. Explicit textures win per channel. FSR-family paths bind a zero transparency texture when omitted and do not generate semantic transparency from color.a.

Which backends support temporal?

Android Vulkan/OpenGL ES, Apple Metal, Windows Vulkan / Direct3D 11 / OpenGL 4.3. CPU temporal is unavailable.

Temporal Super-Resolution

v2.1.0 provides temporal super-resolution through the unified public C ABI: libmagic_sr.a (Windows: libmagic_sr.lib) + interface/mc_interface.h + MC_Enable / MC_Disable. A single function, MC_Enable, handles implicit handle initialization, dynamic reconfiguration, temporal execution, and execution status query.

Modes

Value	Enumerator	Role
0	SPATIAL_SPEED_MODE	Spatial, throughput first
1	SPATIAL_BALANCED_MODE	Spatial, quality/cost trade-off
2	TEMPORAL_SPEED_MODE	Temporal, throughput first
3	TEMPORAL_BALANCED_MODE	Temporal, canonical FSR-family path

- Temporal scaler_factor is (1, 8]; 1.0 itself is rejected. Spatial remains [1.0, 8.0].
- TEMPORAL_SPEED_MODE on Metal, Vulkan, and OpenGLS currently uses a lightweight Convert → Activate → Upscale pipeline. Direct3D 11 and desktop OpenGL 4.3 Speed use the FSR-family temporal path (same family as Balanced, not a separate customer API).
- TEMPORAL_BALANCED_MODE uses the canonical FSR-family path on every GPU temporal backend. Desktop Balanced (Windows Vulkan / Direct3D 11 / desktop GL; Apple Metal) can run edge-resolve; if enable_sharpening is on, RCAS runs after edge-resolve.
- CPU backends (X86 / NEON) cannot run temporal modes.
- The API accepts (1, 8]. Exact 2× has optimized kernels. Not every scale or device is claimed as device-run.
- Internal pipeline names are implementation detail, not public configuration.

Minimum per-frame inputs

- magic_frame_t.image_in / image_out: caller-owned color and output GPU resources.
- magic_frame_t.frame: non-NULL temporal_frame_t* with struct_size = sizeof(temporal_frame_t).
- temporal_frame_t.depth and motion at input resolution.
- jitter_offset_x/y in input pixels, top-left, +X right, +Y down.
- frame_index / reset_history; camera_near / camera_far / camera_fov_y (radians).
- gpu_context at initial MC_Enable: Vulkan requires physical_device + device; D3D11 requires device; Metal device may be NULL (system default); GL/GLES use the current context (library never makes current).
- Vulkan temporal requires magic_frame_t.command_buffer to be a recording VkCommandBuffer. Metal command_buffer is optional. D3D11 / GL / GLES leave it NULL.
- Create-stage depth_reversed / depth_infinite / hdr_color on input_param_t (all-zero = conventional-Z, finite far, LDR). Immutable for the handle lifetime.

Motion, depth, and masks

- Motion is current → previous. The library never negates MVs.

- motion_vector_scale_x/y is the only MV numeric conversion. (0,0) defaults to (input_width, input_height). Mixed zero is rejected.
- Depth is GPU device depth in [0, 1]. Linear view-space depth is not accepted.
- reactive / transparency are optional. Explicit caller textures win per channel.
- When omitted, the library derives an approximate mask. That is not a semantic transparency mask.
- Speed on Metal/Vulkan/GLES may derive both missing reactive and an approximate transparency. FSR-family paths (Balanced on all backends; Speed on D3D11/desktop GL) derive missing reactive only and bind an internal zero transparency texture.
- Both temporal modes enable depth+MV history rejection by default.
- Diagnostic only: MAGICSR_TAAU_AUTO_MASK=0 and MAGICSR_TAAU_HIST_REJECT=0 can turn those defaults off. They are not a customer configuration path.

3. Common problems and diagnostics

- MC_GetVersion should report v2.1.0.
- MC_ERROR_INIT_NULL_PARAM: initial call with *handle == NULL received param == NULL.
- MC_ERROR_PROCESS_NULL_HANDLE: handle is NULL or *handle is NULL.
- MC_ERROR_TEMPORAL_NULL_FRAME: magic_frame_t.frame is NULL in temporal mode.
- Missing color/output/depth/motion handles.
- Vulkan command_buffer NULL or layout UNDEFINED.
- Ghosting: reset_history on hard cuts; check MV current→previous and +Y down jitter.